Kinematic Control and Trajectory Tracking

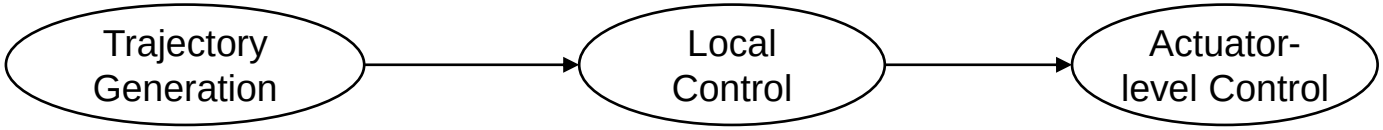
Topics

- Overview of controlling robot motion
  - Actuator, Local, Trajectory Control, Motion Planning, Task Planning
- Control
  - Setpoint v.s. Trajectory Control
  - Open-loop v.s. Closed-loop Control
  - Trajectory Generation

Complementary Reading: J.J. Craig, Introduction to Robotics, Chapter Chapter 7,9.

Controlling Robot Motion Overview

Controlling robot motion is comprises of several stages that from low to high-level goals.



From low-level to high-level:

1. **Actuator-level Control:** how much voltage / current do we supply our motors (or other actuators) to track a desired position for one joint.
2. **Local Control**: how do we determine the joint displacements that will cause the robot to move towards a local goal
   - Sets desired positions for *actuator-level control (1)*
3. **Trajectory Generation**: How do we describe the path that a robot should follow?
   - Sets desired local goal for *local control (2)*

Above Trajectory tracking can be considered **Planning**, comprising:

4. **Motion Planning:** Given a goal that is not “local”, how do I determine the trajectory (path) to the goal.
   - Motion planning describes a path for *trajectory generation (3)*
5. **Task Planning:** Given a task, how do I define a sequence of sub-goals to achieve the task?
   - Sequential pairs of sub-goals are passed as start and end conditions for *motion planning (4)*

<u>Terminology:</u>

- Typically the topic of **control** covers (1)-(3). Special types of control like **optimal control** and **model predictive control** can blur the lines between (3) and (4), because they can solve non-local problems
- Many other interesting control problems, including **adaptive control**, **robust control,** etc.

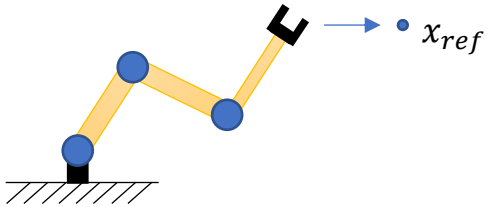
… more in later lectures

Setpoint vs Trajectory Control

Control can be defined as problem of regulating a setpoint (**setpoint control)** or following a trajectory **(trajectory control).**

- **Setpoint control**

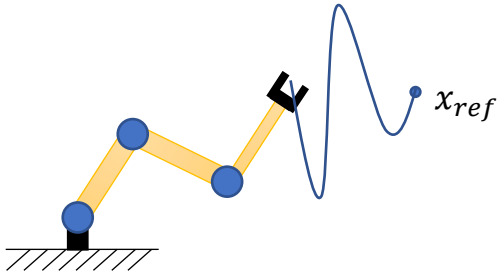
$$x_{ref} = \text{constant}$$



The input to the controller is the end-goal (*setpoint*). A vector is drawn from current position to the setpoint, and the robot moves as quickly as possible to close that gap to the setpoint.

- **Trajectory control**

$$x_{ref}(t) = f(x_{end}, x_0, t)$$



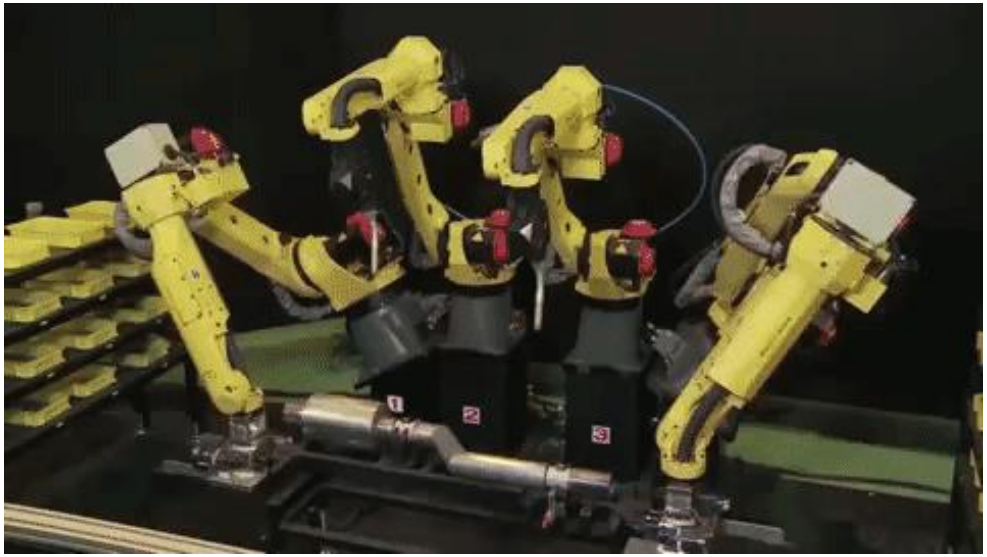
The input to the controller is a *trajectory* described either as a continuous function (rare) or sampled path (typical) that connects the start to the goal.

Examples

Setpoint Control



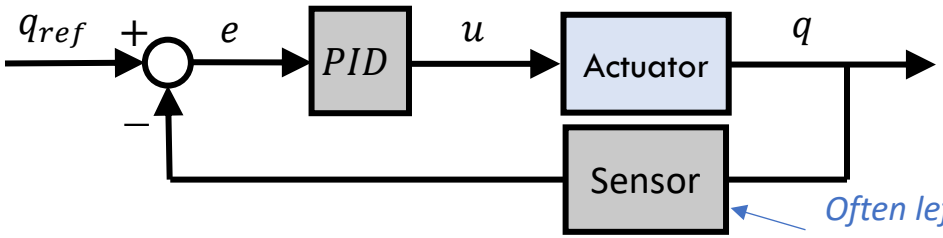
Trajectory Control



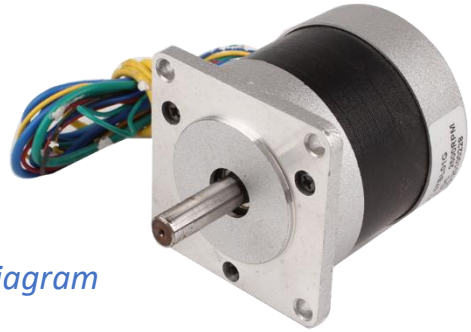
Actuator Control Loop

- Actuators have dynamics associated with them (cannot move instantaneously), so need to wrap a controller around their positions as well.
- Actuator variable $q \in \mathbb{R}$ is measured and can be regulated using a **feedback controller** (or **closed-loop controller**).
- The term to be regulated is the position error $e \in \mathbb{R}$

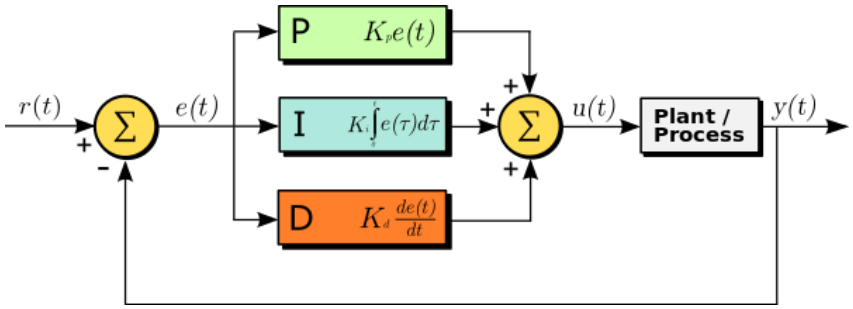
$$e = q_{ref}(t) - q(t)$$



Often left out of block diagram assumed present.



Proportional – Integral – Derivative (PID) Controller



- Used in almost all actuator and local control applications.

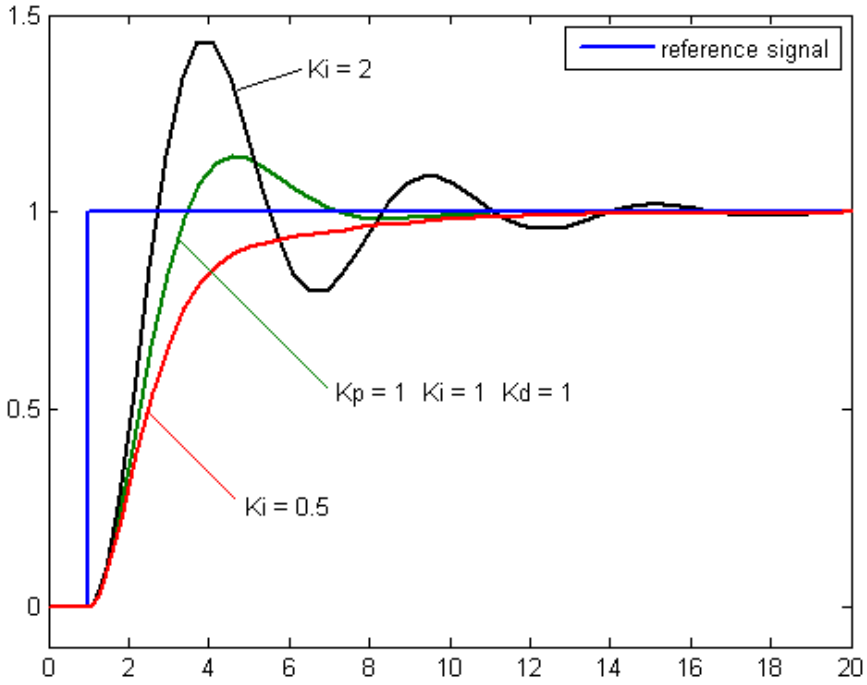
Continuous: $u(t) = K_p e(t) + K_i \int^t e(t)dt + K_d \dot{e}(t)$

Discrete: $u[k] = K_p\big(q_{ref}[k] - q[k]\big) + K_i \sum_{i=0}^{k}\big(q_{ref}[k] - q[k]\big) \ldots$
$+K_d\Big(\big(q_{ref}[k] - q[k]\big) - \big(q_{ref}[k-1] - q[k-1]\big)\Big)$

- K_p, K_i, K_d are constants
- u typically input to actuator (i.e. current or voltage)

Intuition on PID control:

- Controlled spring-like behavior pulling x closer to x_{ref} with K_p
- Controlled damping with K_d
- Controlled zeroing of error with K_i



Tuning Strategy:

- *Manual Tuning Procedure*:

  (1) tune P to slightly overshoot

  (2) tune D to remove overshoot

  (3) tune I only if needed, accepting some level of overshoot

- *Ziegler-Nichols* (look up tables)

Challenges with integral term:

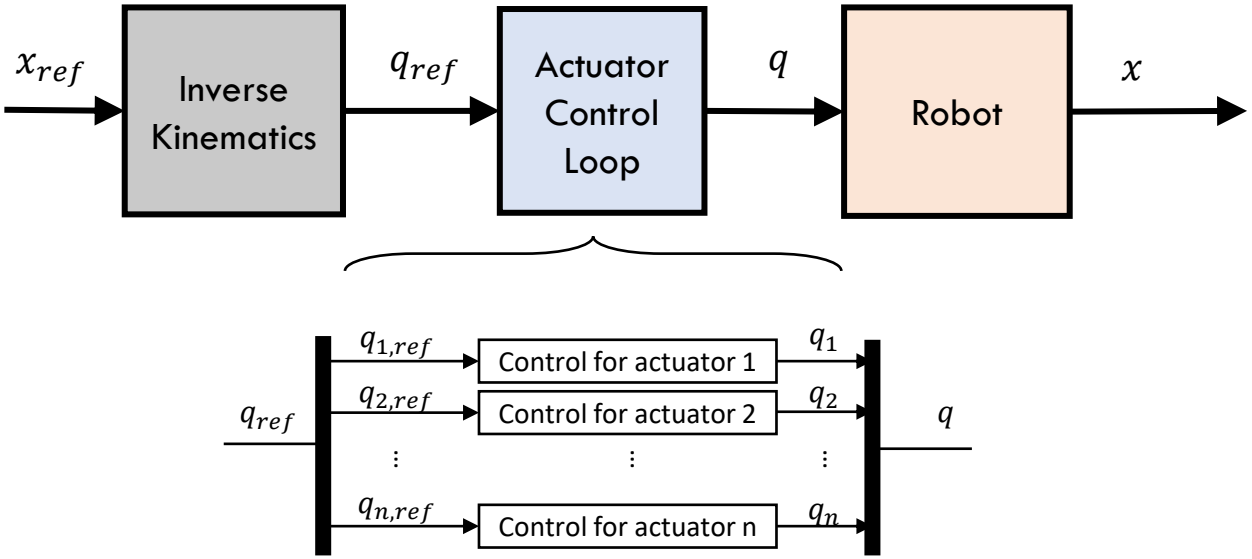
- Accumulation of error $e_i(t) = \int e(t)dt$ is problematic (will never go away)
  - Apply some clamp so that "almost zero" is good enough:
    $e_i(t) = \int e(t)dt$ if $\|e_i(t)\| > eps$
    $e_i(t) = 0$ otherwise
  - Apply some clamp so that e_i doesn't explode:

$$e_i(t) = \min\left(e_{i,max}, \int e(t)dt\right)$$

  - Apply a decay to e_i

$$e_i(t) = \int \gamma^t e(t)dt$$

Local Control: *Open-Loop Setpoint* Control



High level premise:

Assume you have an accurate model of your robot $x = f(q)$ such that by setting q you can expect x to match nearly-perfectly to real life.

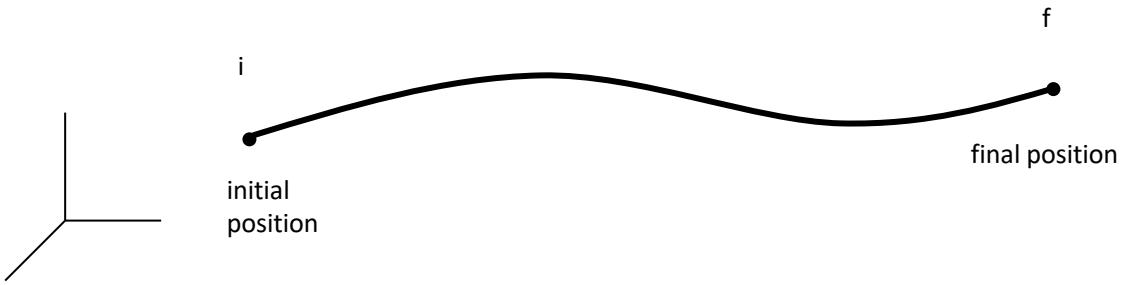
Therefore, given x_{ref} to track,

1. Use inverse kinematics on x_{des} to find q_{des}
2. Move joint i to q_i $\forall i$ under some single-joint-angle control loop.

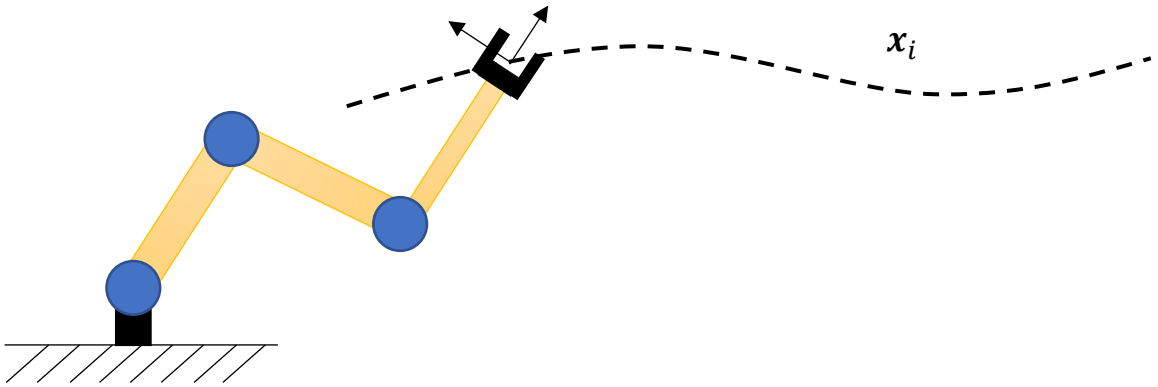
Local Control: *Open-Loop Trajectory* Control

Similar to setpoint control, but now x is a function of time.

1. Given a trajectory $x(t)$ in task space

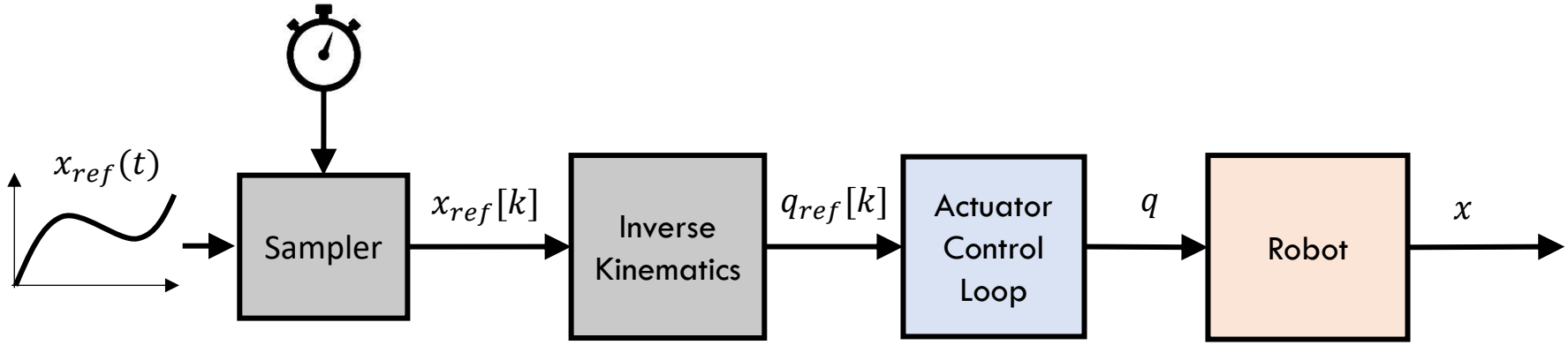


2. Interpolate Trajectory into small segments, $x(t) \rightarrow x_i$ for $i = 1 \ldots K$



For open-loop trajectory control,

1. $x_{ref}(t)$ is sampled at time t_k, i.e. $x[k]$
2. Use inverse kinematics on $x[k]$ to find $q[k]$
   - Can use Jacobian inverse or seed an IK solver with $q[k-1]$ to find $q_{ref}[k]$
3. Move joint i to q_i $\forall i$ under some single-joint-angle control loop.

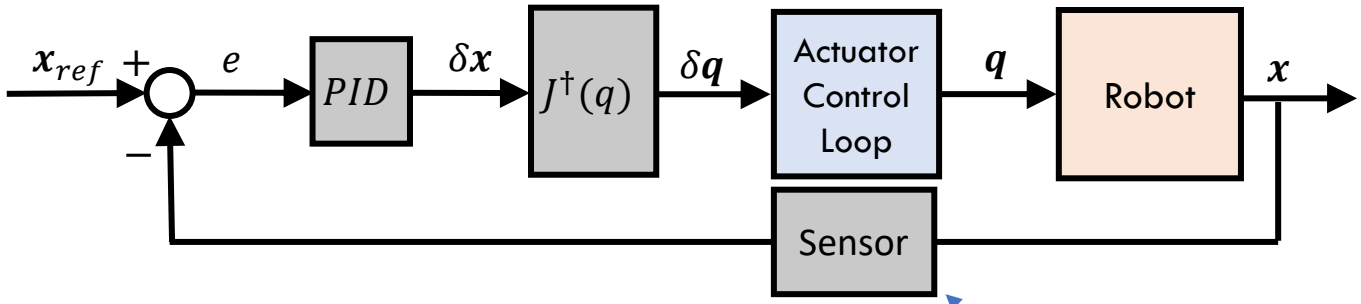


Remarks on *open-loop* control strategies:

- No feedback, so no idea if you are moving correctly towards the Cartesian setpoint
- You must have a very accurate model $x = f(q)$
- Works for high-rigidity robots

Local Control: *Closed-Loop Setpoint* Control

Presumes that you have a way to measure your output cartesian state x so it can correct for model inaccuracies or disturbances in your system.



Often left out of block diagram, assumed present.

Therefore, given $x_{ref} \in \mathbb{R}^6$ to track,

1. Find $e = x_{ref} - x$ to determine error from target
2. Convert e to a desired step towards target
   e.g., assume just a P-controller. Then, $\delta x = \mathrm{K}\Delta x$ where

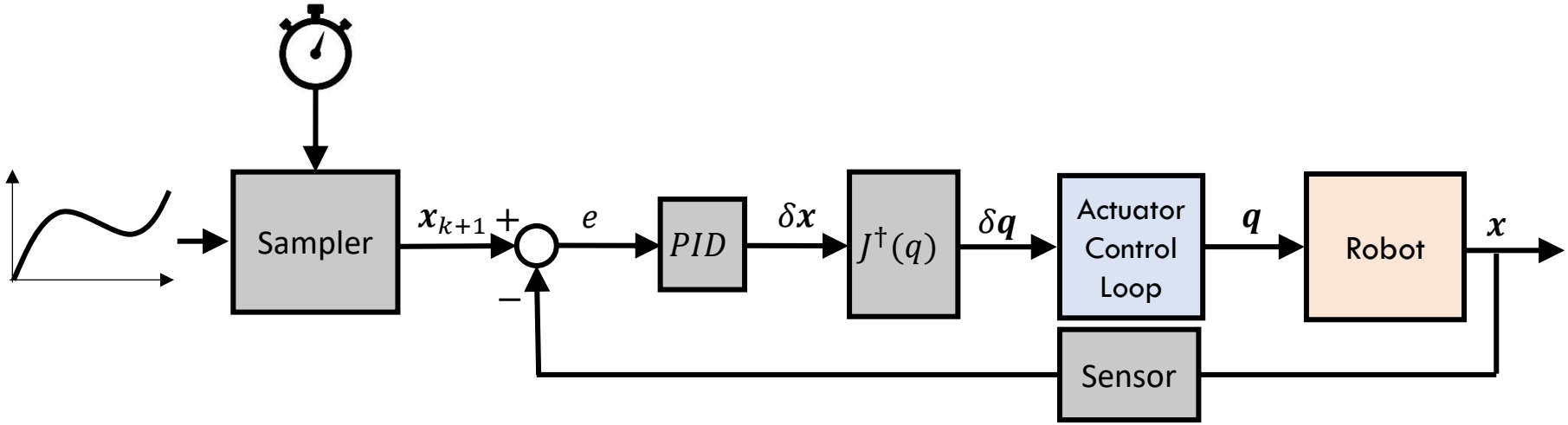
$$K = diag(k_x, k_y, k_z, k_r, k_p, k_y)$$

3. Calculate $\delta q = J^{\dagger}\delta x$, and move actuators δq amount

Local Control: *Closed-Loop Trajectory* Control

Also called **Resolved Motion Rate Control**

1. Measure your current position x
2. For the desired position x_{k+1} at time $t = k$, find the step (error) $\delta x = x_{k+1} - x$.
3. Compute $J(q)$ and its pseudoinverse, $J^{\dagger}(q)$ for the current q.
4. Take a step $\delta q = J^{\dagger}(q)\delta x$ in your actuators to move x towards x_{k+1}.
5. Let $k = k + 1$. Repeat step 2.



Interpolation for Orientation Trajectories

- **Interpolation of a position vector is trivial**
  - Given p_1, p_2, interpolate over each axis x, y, z
  - e.g. $p_1 \rightarrow p_2$ over 10 steps

$$p(k) = p_1 + \frac{k}{10}(p_2 - p_1) \qquad k = 1 \ldots 10$$

  - e.g. over any increment

$$p(k) = (1 - t)p_1 + tp_2 \qquad t \in [0,1]$$

- **Interpolation of between two different orientations is non-trivial**
  - Given θ_1, θ_2 in Euler angles (e.g. z-x'-z"), interpolation of orientations involves walking on a unit ball in SE(3).
  - Path is not a straight line but curved, so linear interpolation produces incorrect results.
- Furthermore, range$(\theta_1, \theta_2) = [0,2\pi]$, subtraction may yield values outside of range. This is called the **wrap-around** problem.

Another problem: euler angles suffer from a problem called gimbal lock.

- It is possible to have a scenario where, if the Euler form is used,

$$R = R_{(1)}R_{(2)}R_{(3)}$$

the *effective* axes of rotation that is caused by performing $R_{(1)}$ and then later $R_{(3)}$ become aligned, resulting in a loss of control in a different axis.

- Consequently, tracking a rotation *through* or *about* the lost rotational direction is not possible.
- Movement becomes very strange at and close to gimbal lock.
- Attempts to track it (i.e. interpolate using just euler angle form) results in loss in tracking
- Gimbal lock is **not a problem for visualization**, but **is a problem for interpolation.**

Quaternion

Representation of orientations in 4-element vector **for interpolation purposes.**

$$\boldsymbol{q} = q_w + q_x i + q_y j + q_z k$$

- Given an axis-angle form of orientation,

$$\begin{aligned} q_w &= \cos(\theta/2) \\ q_x &= u_x \sin(\theta/2) \\ q_y &= u_y \sin(\theta/2) \\ q_z &= u_z \sin(\theta/2) \end{aligned}$$

Quaternion to Orientation Matrix

$$R(\boldsymbol{q}) = \begin{bmatrix} 1 - 2q_y^2 - 2q_z^2 & 2q_xq_y - 2q_zq_w & 2q_xq_z + 2q_yq_w \\ 2q_xq_y + 2q_zq_w & 1 - 2q_x^2 - 2q_z^2 & 2q_yq_z - 2q_xq_w \\ 2q_xq_z - 2q_yq_w & 2q_yq_z + 2q_xq_w & 1 - 2q_x^2 - 2q_y^2 \end{bmatrix}$$

Euler (Roll-Pitch-Yaw, or X-Y-Z) to Quaternion

$c_{(\cdot)} = \cos(\cdot) \quad s_{(\cdot)} = \sin(\cdot)$

$$q_w = \frac{\sqrt{1 + c_\gamma c_p + c_\gamma c_r - s_\gamma s_p s_r + c_p c_r}}{2}$$

$$q_x = \frac{\left(c_p s_r + c_\gamma s_r + s_\gamma s_p c_r\right)}{4w}$$

$$\mathrm{q}_y = \frac{\left(s_\gamma c_p + s_\gamma c_r + c_\gamma s_p s_r\right)}{4w}$$

$$\mathrm{q}_z = \frac{\left(-s_\gamma s_r + c_\gamma s_p c_r + s_p\right)}{4.0}$$

Quaternion to Euler (Roll-Pitch-Yaw, or X-Y-Z)

$$r = \arctan2\left(2q_y q_w - 2q_x q_z, 1 - 2q_y q_y - 2q_z q_z\right)$$

$$p = \arctan2\left(2q_x q_w - 2q_y q_z, 1 - 2xx - 2q_z q_z\right)$$

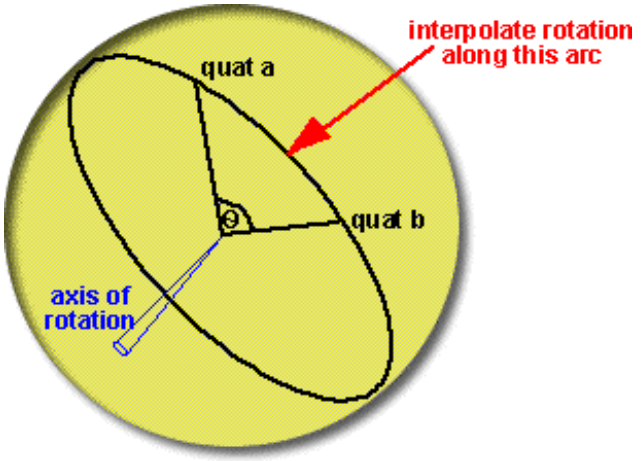
$$\gamma = \arcsin(2q_x q_y + 2q_z q_w)$$

Interpolation Quaternions (SLERP)

Spherical Linear Interpolation (SLERP):

uniform angular velocity around a fixed rotation axis

- Given q_a, q_b, and angle θ between them,



$$\boldsymbol{q_t} = SLERP(\boldsymbol{q}_a, \boldsymbol{q}_b\ , t) = \frac{\boldsymbol{q}_a \sin\left((1-t)\frac{\theta}{2}\right) + \boldsymbol{q}_b \sin\left(t\frac{\theta}{2}\right)}{\sin\left(\frac{\theta}{2}\right)}$$

$$t \in [0,1]$$

Explanation: Linear interpolation is

$$\boldsymbol{x}_t = LERP(\boldsymbol{x}_a, \boldsymbol{x}_b, t) = (1-t)\boldsymbol{x}_a + t\boldsymbol{x}_b$$

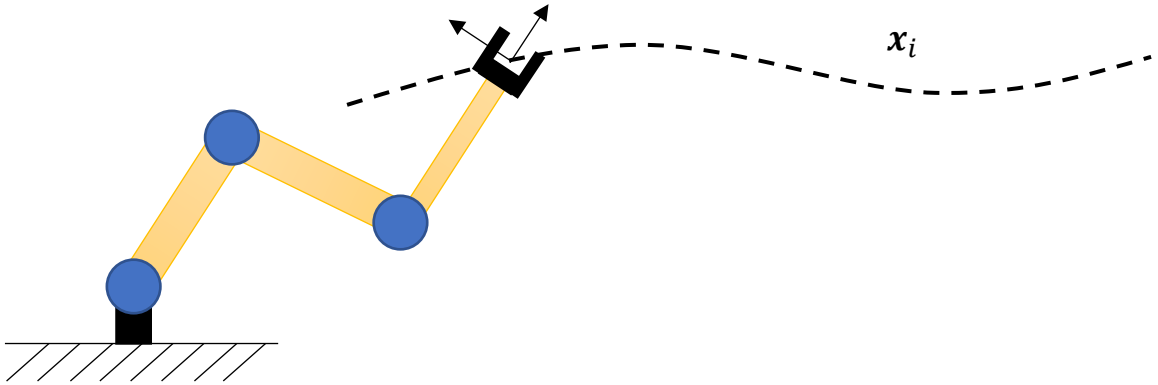
$$t \in [0,1]$$

$SLERP$ just provides the geodesic interpretation of linear interpolation.

Tips with using Quaternions

- Since quaternions are not easy to visualize but easy to correctly perform interpolation of orientations in all situations.
- Euler angles are easy to visualize but difficult to correctly perform rotations in all situations.
- Thus, *use quaternions internally when doing interpolation*, and *expose Euler angles* for visualization or when interfacing with users.

Trajectory Generation



Trajectory Generation takes a sparsely set of defined goals and converts them into a sequence of steps that follows a smooth curve.

- Thus also sometimes called **trajectory interpolation**
- Trajectory generation can be applied at several levels:
  - Actuator control $q_i[k] \rightarrow q_k[k+1]$
  - Local control $x[k] \rightarrow x[k+1]$

Connecting between two setpoints (or waypoints)

- Very simple premise: fit positions and velocities to **polynomials**
- Solve a **two-point boundary value problem** (initial and final conditions)

  e.g. move-and-stop: $v(0) = 0, v(T) = 0$

Let p_0, v_0 and p_f, v_f represent initial and terminal conditions, then a position trajectory satisfying both conditions becomes:

$$p(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

$$v(t) = a_1 + 2a_2 t + 3a_3 t^2$$

Solve:

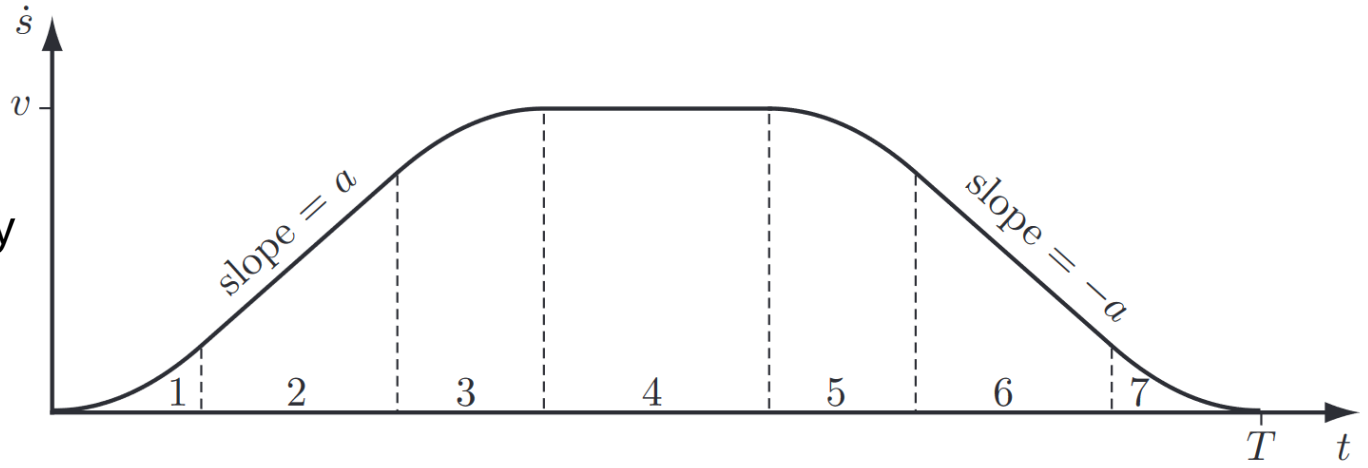
$$\begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 \\ 0 & 1 & 2t_0 & 3t_0^2 \\ 1 & t_f & t_f^2 & t_f^3 \\ 0 & 1 & 2t_f & 3t_f^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} p_0 \\ v_0 \\ p_f \\ p_f \end{bmatrix}$$

Invert matrix to find $a_0 \ldots a_3$

Easy to expand to more conditions (e.g. acceleration, jerk)

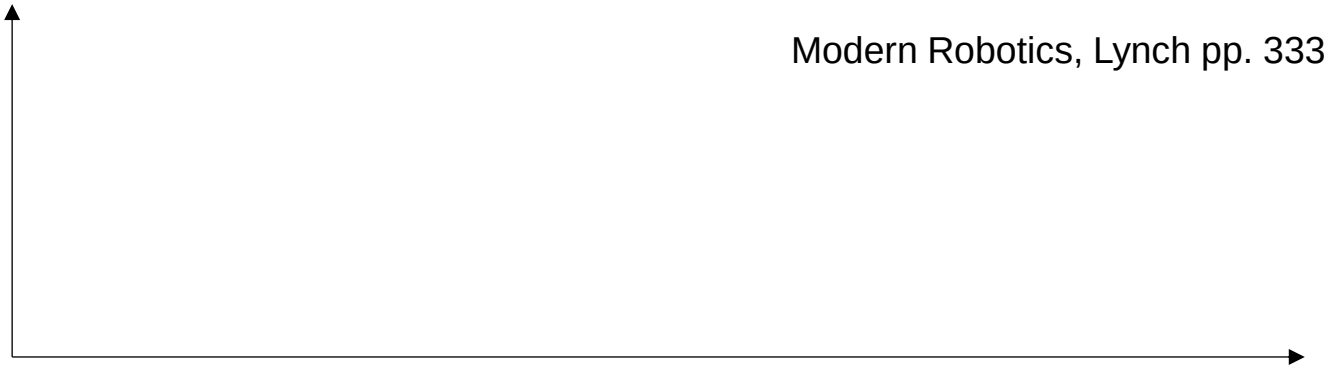
Can construct more complicated trajectories

Velocity



Modern Robotics, Lynch pp. 333

Position



Acceleration



Discretization:

- Given a path $x(t)$ or $q(t)$ you can now resample the path based on your control loop rate.

  e.g. Given $v(t)$ defined by the trajectory, resampling can be defined as

$$v[k] = v(k\Delta t) \qquad k = 1 \ldots K$$

$$\Delta t = \frac{1}{\text{control frequency}}$$

  … solve for k

Multiple Waypoints: Given a trajectory with multiple waypoints, how do you do trajectory interpolation?

- Combine in piece-wise manner to create **splines**.
- Many different spline types, depending on if waypoints are non-uninformly spaced, if higher-order derivative boundary conditions need to be met, etc.

